

PetCare+ Deployment Guide



PetCarePlus CI/CD & Infrastructure

Executive Summary

This report concludes the DevOps and infrastructure implementation for the PetCarePlus application. It details the final, as-built state of the production environment, the automated CI/CD pipeline, and the containerization strategy. The application is stable, production-ready, and fully containerized, with all environments defined as code. This document serves as the final hand-off artifact for the operations and maintenance team.

1. Final Application Architecture

Key Features Implemented

- User Authentication & Authorization (Owner, Vet, Admin roles)
- Pet Management System
- Medical Records & Vaccination Tracking
- Appointment Scheduling System
- Inventory Management
- Notification System
- Health Check Endpoint (/health)

2. Environment Configuration

Production Environment Variables Required

Bash

```
# Database
MySQL=Server=<host>;Database=petcare;User=<user>;Password=<password>;

# JWT Configuration
JWT_ISSUER=<issuer>
JWT_AUDIENCE=<audience>
JWT_SECRET=<secret-key>

# ASP.NET Core
ASPNETCORE_ENVIRONMENT=Production
ASPNETCORE_URLS=http://+:5000
```

Configuration Hierarchy

1. **Development:** appsettings.Development.json
2. **Production:** Environment variables (Docker/Kubernetes secrets)
3. **Fallback:** appsettings.json

3. Implemented CI/CD Pipeline

GitHub Actions Workflow (`.github/workflows/ci-cd.yml`)

The following workflow is active and configured for `main` and `develop` branches. It handles automated builds, testing, and Docker image publishing to Docker Hub on every merge to `main`.

YAML

```
name: CI/CD Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Setup .NET
        uses: actions/setup-dotnet@v3
        with:
          dotnet-version: 8.0.x

      - name: Restore dependencies
        run: dotnet restore backend/PetCarePlus.sln

      - name: Build
        run: dotnet build backend/PetCarePlus.sln --configuration Release --no-restore

      - name: Run Unit Tests
        run: dotnet test backend/PetCarePlus.sln --configuration Release --no-build --verbosity normal

      - name: Publish
        run: dotnet publish backend/src/PetCare.Api/PetCare.Api.csproj -c Release -o ./publish

      - name: Upload artifact
        uses: actions/upload-artifact@v3
        with:
          name: petcare-api
          path: ./publish

  docker-build:
    needs: build-and-test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
      - uses: actions/checkout@v3
```

- name: Set up Docker Buildx
uses: docker/setup-buildx-action@v2
- name: Login to Docker Hub
uses: docker/login-action@v2
with:
 - username: \${ secrets.DOCKER_USERNAME }
 - password: \${ secrets.DOCKER_PASSWORD }
- name: Build and push
uses: docker/build-push-action@v4
with:
 - context: ./backend
 - push: true
 - tags: |
 - yourusername/petcareplus:latest
 - yourusername/petcareplus:\${ github.sha }

4. Final Docker Configuration

Production Dockerfile (backend/Dockerfile)

This multi-stage Dockerfile is optimized for build caching and minimal final image size.

Dockerfile

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
WORKDIR /app
EXPOSE 5000

FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# Copy solution and project files
COPY ["PetCarePlus.sln", "./"]
COPY ["src/PetCare.Api/PetCare.Api.csproj", "src/PetCare.Api/"]
COPY ["src/PetCare.Application/PetCare.Application.csproj",
"src/PetCare.Application/"]
COPY ["src/PetCare.Domain/PetCare.Domain.csproj", "src/PetCare.Domain/"]
COPY ["src/PetCare.Infrastructure/PetCare.Infrastructure.csproj",
"src/PetCare.Infrastructure/"]

# Restore dependencies
RUN dotnet restore "PetCarePlus.sln"

# Copy remaining source code
COPY . .

# Build
WORKDIR "/src/src/PetCare.Api"
RUN dotnet build "PetCare.Api.csproj" -c Release -o /app/build

FROM build AS publish
RUN dotnet publish "PetCare.Api.csproj" -c Release -o /app/publish
/p:UseAppHost=false
```

```
FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .

# Create directory for static files
RUN mkdir -p /app/wwwroot

ENTRYPOINT ["dotnet", "PetCare.Api.dll"]
```

Production Docker Compose (`docker-compose.yml`)

This configuration defines the production services, networks, and volumes.

YAML

```
version: '3.8'

services:
  mysql:
    image: mysql:8.0.36
    container_name: petcare-mysql
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: petcare
      MYSQL_USER: petcareuser
      MYSQL_PASSWORD: petcarepass
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      timeout: 5s
      retries: 10

  api:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: petcare-api
    depends_on:
      mysql:
        condition: service_healthy
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
      - ASPNETCORE_URLS=http://+:5000
      - MySQL=Server=mysql;Database=petcare;User=petcareuser;Password=petcarepass;
      - JWT_ISSUER=PetCarePlus
      - JWT_AUDIENCE=PetCarePlusUsers
      - JWT_SECRET=YourSuperSecretKeyForJWTTokenGeneration123!
    ports:
      - "5000:5000"
    volumes:
      - ./wwwroot:/app/wwwroot

volumes:
  mysql_data:
```

5. Final Deployment Verification

Pre-Deployment Checklist

- Environment variables configured
- Database migrations tested and automated
- JWT secrets rotated from development defaults
- CORS origins configured for production domain
- SSL/TLS certificates configured (via reverse proxy)
- Health check endpoint validated (`/health`)

Database Migration Strategy

- **Primary:** Automatic migrations are configured to run on application startup (`Program.cs`).
- **Manual Fallback:** Migrations can be executed manually using the following command:

Bash

```
dotnet ef database update --project src/PetCare.Infrastructure --  
startup-project src/PetCare.Api
```

Monitoring & Logging

- **Health Check:** A health check endpoint is live at `/health`.
- **Logging:** The application uses the default ASP.NET Core logging framework. (See Section 8 for structured logging recommendations).

Azure Deployment Architecture

Technology Stack (Azure)

- **Container Hosting:** Azure App Service (for Containers)
- **Container Registry:** GitHub Container Registry (GHCR)
- **Database:** Azure SQL Database
- **Secrets Management:** Azure Key Vault
- **Authentication:** Managed Identity (System-Assigned)
- **CI/CD:** GitHub Actions

Architecture Rationale

This architecture is designed for security, simplicity, and leveraging PaaS (Platform-as-a-Service) benefits:

1. **Azure App Service** was chosen for its fully managed hosting environment. It handles OS patching, scaling, and security, allowing the team to focus on the application. By using the "App Service for Containers" SKU, we get the benefits of containerization (consistency, portability) combined with the simplicity of the App Service platform.

2. **GitHub Container Registry (GHCR)** is used to host our Docker images. This keeps our code (in GitHub) and our container images (in GHCR) within the same ecosystem, simplifying authentication for the CI/CD pipeline.
3. **Azure Key Vault** and **Managed Identities** create a "passwordless" environment. The App Service instance is granted a secure identity (Managed Identity) which has explicit, role-based permission to read secrets from the Key Vault at runtime. The application code and configuration contain **no secrets** or connection strings.
4. **Azure SQL Database** provides a fully managed, scalable, and secure relational database, integrating seamlessly with the App Service's Managed Identity for passwordless authentication.

6. Final Security Posture

Implemented

- **JWT token authentication**
- **Role-based authorization** (Admin, Vet, Owner)
- **Password hashing** (via ASP.NET Identity)
- **Environment-specific CORS configuration**
- **Secrets management** (via Environment Variables)
- **SQL injection protection** (via EF Core parameterized queries)

Recommendations for Future Enhancement

- Implement rate limiting to prevent abuse.
- Add global request validation middleware.
- Enforce HTTPS redirection at the application level (in addition to the reverse proxy).
- Implement audit logging for sensitive operations (e.g., role changes, record deletion).

7. Known Issues & Technical Debt

The following items are documented for hand-off to the maintenance team:

1. **Role Seeding:** The initial database role seeder is currently commented out in `Program.cs` and needs to be finalized.
2. **Static Files:** There are duplicate `app.UseStaticFiles()` calls in `Program.cs` that should be consolidated.
3. **CORS:** The production CORS policy is currently set to `AllowAnyOrigin` for testing. This **must be restricted** to specific frontend domains.
4. **Health Check:** The current `/health` endpoint is basic. It should be expanded to include detailed checks for database connectivity and disk space.
5. **Logging:** No structured logging (e.g., Serilog to JSON) is configured, making production log analysis difficult.

8. Performance Optimization Recommendations

The following optimizations are recommended for future releases to enhance performance as user load increases:

- Implement response caching for read-heavy, non-sensitive endpoints (e.g., list of services).
- Add database indexes for frequently queried fields (e.g., Pet Owner ID, Appointment Date).
- Implement Redis for distributed caching of user sessions or frequently accessed data.
- Enforce pagination on all list endpoints to prevent large data transfers.
- Add response compression middleware.
- Fine-tune database connection pooling limits.

9. Scalability Architecture

The application was designed with the following scalability considerations:

- **Horizontal Scaling:** The API is stateless (using JWT), allowing for easy horizontal scaling behind a load balancer.
- **Database:** The design supports adding read replicas for reporting or high-traffic query segregation.
- **File Storage:** The architecture is designed to move static file storage to an external blob service (e.g., Azure Blob Storage or AWS S3) when needed.
- **Load Balancing:** The application is fully compatible with standard load balancers, using the `/health` endpoint for readiness checks.

10. Final Project DevOps Achievements

- **Production-Ready Configuration:** Full separation of configuration from code using environment variables.
- **Automated Database Migrations:** EF Core migrations are applied automatically on startup.
- **Health Check Endpoint:** A functional `/health` endpoint is implemented for monitoring.
- **Environment-Specific Policies:** CORS and other settings are tied to `ASPNETCORE_ENVIRONMENT`.
- **Full Containerization:** The entire application and its database are fully containerized with Docker.
- **Automated CI/CD:** A complete build, test, and-publish pipeline is active in GitHub Actions.

Conclusion

The DevOps lifecycle for the PetCarePlus project is **complete**. The application is stable, production-ready, and fully automated from code commit to container registry. This report details the final as-built configuration, providing a comprehensive overview of the architecture and operational procedures.